

Irreversible Metadata Scrubber — Client Progress Report

Status: Phases 0, 1 and 2 complete · Phase 3 documents — **PDF complete at all three levels**, Word next **Scope of this report:** what has been built, what has been tested and proven, and what comes next

1. Executive summary

Every file you send someone carries hidden information you did not intend to send: where a photo was taken, which phone or program made it, when it was written, who edited it, and often a small preview image or an earlier draft buried in the bytes. This project builds a tool that **removes that hidden information so it cannot be recovered again** — not merely deleted from view, but gone from the file even when someone examines it byte-by-byte with forensic software.

Where we are today. The tool works, ships as a command-line program, and handles **six file formats end to end: JPEG, PNG, MP3, FLAC, M4A and PDF** — PDF now at all three cleaning levels, which is the whole of the hard half of the documents phase. It is backed by **345 automated tests** that run on every code change, and by a measurement system that re-proves every published claim from scratch on each run — if a result stops being true, the build fails.

What makes this different from existing tools. Mature tools already delete tags well; we are at parity there and say so. Our contribution is three things nobody else offers together:

1. **You choose the trade-off.** Three cleaning levels, from "don't touch the picture at all" to "make the file untraceable", so the user picks the strength that matches the risk they actually face.
2. **We defeat the encoder fingerprint** — the hidden signature that reveals *which program or device made a file* even after every tag is gone. For two formats (PNG and FLAC) we now do this at **zero quality cost**: identical pixels, identical audio, down to the last bit.
3. **Every claim is measured, and every limit is published.** We maintain a written register of what the tool cannot do, in plain language, and the automated build publishes it alongside the successes. There is no claim in this project that is not backed by an experiment.

What has happened since the last report. Phase 3 opened on documents, and **PDF is now finished across all three levels**. The single most famous document leak in existence — a PDF's supposedly deleted earlier drafts still sitting inside the published file — is closed by construction, and measured closed with three independent attacks. Two further results came out of it, and both are *negative* results we chose to publish rather than round away: after every tag is gone, **a PDF still reveals which program typeset it**, and **flattening the document to pictures does not fix**

that — we proved it against our own output rather than claiming the win. Those are now published limits, with the reason each one is a floor rather than an unfinished job.

What is next. The rest of Phase 3 — **Word (.docx)**, whose target is a known failure every surveyed competing tool shares: revision-save IDs that survive every scrubber on the market. A new capability has also been specified and scheduled: an optional mode that gives a cleaned file a **plausible cover story** instead of leaving it conspicuously blank (§7).

2. What "irreversible" means here, and who we defend against

The word does a lot of work, so we pin it down precisely.

Irreversible = forensically unrecoverable from the cleaned file itself. Not hidden, not blanked, not overwritten with spaces. If the original file still sits in the user's own folder afterwards, that is a workflow question for the user — not a property of the file we produce.

We defend against a **medium-tier adversary**: a journalist, a competitor, or an amateur investigator using off-the-shelf forensic tools. Not a nation-state intelligence agency. Being explicit about this is what lets us make claims we can actually prove.

That adversary comes in two strengths, and beating the first is easy while beating the second is the real work:

	Who they are	What they read
A1 — the tag snoop	Opens the file and reads what is written in it	GPS coordinates, camera make and model, author name, timestamps, editing software, embedded thumbnails, comments
A2 — the fingerprint snoop	Assumes every tag is already gone, and studies *how* the file was built	The compression settings, the ordering of internal structures, the encoder's habits — enough to say "this came from an iPhone" or "this came from Photoshop" with no tag present at all

A2 is the interesting adversary. It is like identifying a typewriter from the shape of its letters when there is no name on the page. Most privacy tools have no answer to it.

A third level, A3, is where the investigator already holds the *original* file and compares. We name it for completeness; it is outside what any scrubber can defeat.

3. The three cleaning levels

The user chooses how much fidelity to spend on anonymity. This choice is the product.

Level	What it does	What it costs
F1 — bit-preserving	Removes every piece of metadata; the picture or audio is left completely untouched , byte for byte.	Nothing.
F2 — lossless rebuild	Removes metadata and rebuilds the file's packaging through one standard route, so the packaging no longer reveals its origin. Content still perfect.	Nothing — genuinely zero.
F3 — full re-encode	Re-records the picture or sound through one standard encoder, so every file we produce carries the same signature as every other.	A small, generally invisible quality loss.

The tool identifies file types **by their actual contents, not their filename**, and is **fail-closed**: when it cannot fully guarantee a result, it refuses and writes nothing rather than hand back a file that merely looks clean.

```
python -m src.scrub --fidelity F1|F2|F3 <input> <output>
```

4. What has been built

4.1 The foundation (Phase 0)

Before any single file format, we built the shared machinery — this is why later formats land quickly and consistently rather than each being a bespoke effort:

- **A differential-testing harness.** The core verification idea: take two files with identical content but different metadata, clean both, and compare. Anything that still differs is a leak, by definition. This is the pass/fail gate for every format; nothing ships on visual inspection.
- **A test corpus generator** that injects metadata into **every known hiding place** at once, not just the obvious tags — so passing means all duplicates were cleared.
- **Shared standards modules**, written once and called by every format: EXIF/TIFF, XMP, ICC colour profiles, IPTC, ISO base media (the box format behind M4A, MP4 and HEIC), and a perceptual quality gate. Copy-pasting these per format is how tools develop leaks; we deliberately do not.
- **A scrubber-fingerprint guard** — a check that **our own tool** leaves no signature: no "cleaned by" string, no distinctive padding, no timestamps, deterministic output. It has already caught us twice and forced changes.
- **Magic-number dispatch**, so every format plugs into one entry point.

4.2 Images — Phase 1, complete

- **JPEG** at all three levels, including full removal of the embedded thumbnail (a small copy of the photo, with its own separate GPS and camera data — the most commonly missed hiding place in the industry).
- **PNG** at F1 and F2. PNG needs no F3: it is lossless by nature, so **F2 already achieves untraceability at zero cost**.

4.3 Audio — Phase 2, complete

- **MP3** — tag removal at F1, and a canonical re-encode at F3 that erases the encoder fingerprint. Proven against a genuinely **different** encoder, not just a different program driving the same one.
- **FLAC** — F1 and F2, with **untraceability achieved losslessly**: the audio comes out bit-identical, mathematically verified through the format's own internal checksum.
- **M4A** — F1, F2 and F3, including surgery on the file's internal box structure with sample-table repair. **This is the format MAT2, the leading open-source privacy tool, refuses to process at all**. A user who hands MAT2 an `.m4a` gets nothing back; ours handles it at all three levels.

4.4 Documents — Phase 3, PDF complete

PDF is the hardest format handled so far, and the only one where the tool writes every byte of the output itself rather than letting a library save the file.

- **F1** — a rebuild from the document's own object graph, with **our own serializer**. Every earlier draft dies by construction, because only what the finished document actually reaches is written out; there is no deletion step that could miss one. It clears the document information block, XMP metadata **wherever it is attached** (including hanging off an image), the document identifier, page thumbnails, private application data, layer creator info and annotation authors — and it **recurses**: an embedded photo goes through the Phase 1 JPEG handler and comes back with its GPS and camera data gone and its pixels bit-identical. It also reaches images painted directly into the page, which no scan of the file's object list can even see.
- **F2** — a canonicaliser that rewrites every page-painting instruction through one single writer, so five different programs stop speaking five different dialects.
- **F3** — rasterise and rebuild: each page rendered by one renderer at one resolution and reassembled, offered with its real cost stated first (the text stops being selectable, searchable and screen-reader-readable).
- **A redaction-risk advisory** that warns when a document looks like someone tried to black out text by drawing a box over it. It **warns and never repairs**, never fails a scrub, and deliberately under-reports rather than implying safety.
- **Files we refuse outright** rather than half-clean: encrypted, digitally signed, attachment-bearing, XFA-form and internally inconsistent documents.

4.5 The engineering around it

- **19,700 lines** of Python across the tool, harness, experiments and reporting.
- **Continuous integration** on every change: code quality, the full test suite on both ends of the supported Python range, a coverage gate, and — most importantly — a job called `"Published results still true"` that re-measures every published claim from scratch and **fails the build if a result has drifted**. Honesty is enforced by machine, not by memory.
- **A plain-language QA report** published automatically on every run, written for a non-technical reader: a verdict banner, a pipeline diagram coloured by what actually happened, before-and-after evidence on real sample files, and a "what we can't do yet" section rendered directly from our limits register.

5. What has been tested, and what it proved

5.1 Results (measured, never assumed)

Format	A1 — tags removed (F1/F2/F3)	A2 — fingerprint erased (F1/F2/F3)
JPEG	pass / pass / pass	fail / fail / pass
PNG	pass / pass / n-a	fail / pass / n-a
MP3	pass / n-a / pass	fail / n-a / pass
FLAC	pass / pass / n-a	fail / pass / n-a
M4A	pass / pass / pass	fail / narrowed to one channel
PDF	pass / pass / pass	fail at every level — and <i>*why*</i> is the finding

Reading this table: the tag snoop (A1) is defeated everywhere, at every level. The fingerprint snoop (A2) is defeated by rebuilding — and **twice now at no quality cost at all** (PNG and FLAC), while JPEG and MP3 must spend one re-encode to get there.

The dividing line is a real technical finding, not a limitation of effort: if the fingerprint lives in a **separable layer** (PNG's compression, FLAC's metadata blocks) it can be normalised for free. If it is **baked into the compressed content itself** (JPEG's quantization tables, MP3's encoder header and bitrate contour), removing it costs a generation of quality. That distinction is a genuine contribution and it is what the Pareto matrices exist to express.

PDF is the case where the answer is "no", and that is the result. Every tag comes off at all three levels, and the document's edit history is gone. But **which program typeset the page** survives, because it is written in the geometry of the page itself — where each letter sits, where the lines break, which letters were embedded at all. Changing that means re-typesetting the document, which changes what the reader sees, so it is a **floor** in the same sense that camera

sensor noise is a floor. We publish the failing cell with the reason, rather than choosing a weaker test that would pass.

5.2 The experiments behind the table

Each A2 claim comes from a named experiment with a stated method, not from reasoning:

Experiment	The question it answers	Result
E3	Does JPEG's compression signature identify the producing program?	Yes at F1/F2 — erased at F3
E-LAME	Do MP3 file headers still identify the encoder after cleaning?	All producers collapse to one signature
E-ENGINE	Harder: with headers normalised, can the encoder be recovered from the sound itself ?	0.89 → 0.53 at 44.1 kHz, 0.94 → 0.58 at 22.05 kHz (chance = 0.50)
E-FLAC	Can FLAC be made untraceable without touching the audio?	Yes — bit-identical audio, verified by checksum
E-M4A-AUDIO	Does the original recording's quality setting survive in the sound?	0.88 on untouched files → chance (0.50) after cleaning
E-PDF-HISTORY	Do a PDF's earlier drafts survive — ours, and the standard tools'?	Ours: nothing recoverable, under three separate attacks. The most widely used tool: removes nothing
E-PDF	Which half of a PDF still names the program that made it — how the file was built, or how the page was typeset?	The build half is closed at F1. The typesetting half is half-closed at F2 and named as a floor
E-PDF-RASTER	After a PDF is flattened to pictures, do the pixels still name the typesetter?	Yes — every page, at every resolution we tried, down to unreadable text

Two methodological commitments run through all of them, and both were adopted after they caught us:

- **Verdicts are statistical significance tests, not thresholds.** An earlier version used a fixed accuracy margin; with a small sample that turned random noise into a verdict, and the same experiment landed on opposite answers on two different machines. We enlarged every peer set until the answer stopped moving and now report p-values.
- **Every experiment asserts its own controls.** If an attack cannot identify the producer of an *untouched* file, its failure on a cleaned one proves nothing — that case is reported as **inconclusive**, never as a pass. We apply this against our own interest: the M4A engine-identity question is reported as *unanswerable with this corpus* rather than counted as a win.

5.3 How we compare to the tools that already exist

Measured directly against ExifTool, MAT2 and jpegtran on our own torture corpus:

Capability	Ours	ExifTool	MAT2	jpegtran
Removes all named metadata	✓	✓	✓	✓
Keeps the picture pixel-perfect	✓	✓	✗ always lossy	✓
No cumulative damage when run repeatedly	✓	✓	✗ degrades each pass	✓
Erases the encoder fingerprint (untraceable)	✓	✗	⚠ only as a side effect	✗
Lossless *and* untraceable	✓	✗	✗	n/a
You choose the fidelity for the threat you face	✓	✗	✗	✗
A measured guarantee matrix	✓	✗	✗	✗
Handles M4A audio at all	✓	⚠ tags only	✗ refuses	n/a
Clears a FLAC third-party data block	✓	⚠ not by default	✗ leaves it	n/a

Two concrete findings worth naming. Running the leading tools on our corpus, **MAT2 leaves an arbitrary third-party data block inside FLAC files completely untouched** — a general-purpose container any application can write anything into, forwarded by a privacy tool that never inspected it. And **ExifTool, run on an M4A, leaves the file's creation and modification times behind**, because they are structural fields rather than tags.

Equally, the honest half: on plain tag removal from a normal JPEG, we are at **parity** with mature tools, not ahead. That is reported as parity.

5.4 The limits we publish

We maintain a register of **eighteen** known limits in plain language, and the automated report renders it verbatim on every run — a limit recorded there reaches every reader automatically. The headline ones:

- **A cleaned file is recognisably cleaned.** Everything we produce comes out in the same standard shape, so an observer can tell a scrubber was used. They cannot tell **which** program, phone or camera made the original — and that second question is the one this tool promises to answer. Hiding that a file was cleaned is a different problem.
- **Camera sensor noise (PRNU) and microphone/room signatures cannot be removed** by any metadata tool that exists. They live in the picture and the sound themselves. We document these as structural impossibilities rather than pretending they are solved.

- **A more sophisticated investigator than the one we built might still succeed.** Our audio attack studies the frequency profile; published research also studies the encoder's internal arithmetic, which we have **not** tested against. We name the gap rather than implying it away.
 - **Some unusual audio files, and several classes of PDF, we refuse outright** rather than half-clean them. Nothing leaks, but the user gets no output — broader support is planned. A signed document is the clearest case: any rewrite destroys the signature, and handing back a silently broken one is worse than saying so.
 - **A PDF still names the program that typeset it, at every level**, and flattening it to pictures does not change that — measured, published as a failing result, with the reason it is a floor rather than a gap.
 - **Blacking out text in a PDF is not removing it.** The words stay in the file underneath the box. This is a different problem from the one this tool solves; we **warn and never silently repair**, and a test makes sure the warning stays necessary.
-

6. Phase 3 — documents: what PDF now does, and what it proved

This is the most valuable phase in the plan, because documents are where the real-world disclosures happen. The PDF half is complete; the Word half is next.

6.1 The leak this phase exists to close, and the proof it is closed

A PDF is edited by appending. The old version stays in the file underneath the new one — text "deleted" three revisions ago is still there in full, and cutting the file at an earlier stopping point opens the earlier draft as a normal document. This is the mechanism behind the famous cases where published, "redacted" documents were read straight through the black boxes.

Our answer is not to delete the history but to never copy it. The clean rebuilds the document from what the finished version actually references, so a superseded draft is never written out in the first place. There is no deletion pass that could miss one.

We then measured it three separate ways rather than trusting the argument: rolling the file back to an earlier stopping point, carving the raw bytes for text that was deleted in an earlier draft, and auditing every object in the file to separate **replaced** content from **abandoned** content. Nothing recoverable, under all three.

What the standard tools do on the same document, measured rather than repeated from their documentation:

Tool	Result
The most widely used metadata tool	Removes nothing. It also edits by appending, so it adds a <i>*fourth*</i> revision and leaves all three earlier drafts intact — it warns about this itself
The leading open-source privacy tool	Destroys the history, but then clears the metadata by appending an update of its own — leaving its own name and a wall-clock timestamp including the operator's timezone one layer down

Reproduced on both of that tool's paths and on a real 295 KB document.

6.2 The second finding: what a cleaned PDF still gives away

With every tag gone, we asked the harder question — can you still tell which program made the document? We measured it across **five real producers printing the same document** (Chrome, LibreOffice, macOS Preview, and two synthetic producers built to differ deliberately), and deliberately measured it in **two halves** rather than as one verdict, because the two halves have completely different prospects:

- **How the file was built** — object ordering, the index format, the identifier, the binary marker in the opening line. **Closed outright at F1.** After our clean, none of the five producers can be told apart on any of it.
- **How the page was typeset** — the instructions that paint the text. **Half-closed at F2:** rewriting every instruction through one writer makes four of the five speak an identical language, and two producers that differed only in **how** they wrote the same page become indistinguishable. The pages still render **byte-identical** to the originals — verified by rendering both to images, not by trusting our own rule.
- **Where the letters physically sit** — line breaks, spacing, which letters were embedded. **Not closable.** It cannot change without re-typesetting the page, which would change what the reader sees.

Measuring **before** designing the fix is the point of this ordering: it is what separates a genuine floor from unfinished work.

6.3 The third finding: we disproved our own escape route

The obvious way out is to flatten the document to pictures — the technique the leading privacy tools use. Structurally it passes almost trivially, since the file is then entirely our own output, and **publishing that as a win would have been exactly the overclaim this project exists to avoid.**

So we attacked the pictures instead. A classifier reading only the shape of the ink identifies which of the five producers made a page **100% of the time** (30 pages, five producers, one-in-five by chance), against a valid control on unscrubbed pages.

We then tried the obvious mitigation, and **the measurement corrected our own prediction.** We expected coarser rendering to blur the signal, on the assumption it lived in fine sub-pixel detail. It does not: swept from 300 DPI down to **18 DPI, where the body text is unreadable, the**

classifier stays at 100%. Taking the feature vector apart says why — overall ink density alone is at chance, while the **column profile alone reaches 100%:** the margins and the width of the text block, which downsampling leaves completely intact. **There is no resolution at which the document is still useful and the typesetter is anonymous.**

6.4 What else came out of it

- **The fingerprint guard failed the moment F2 landed, and was right to.** We were compressing every stream at maximum effort, which no real producer does — four of the five use the middle setting. Maximum effort did not normalise our output, it *labelled* it. We now use the setting the largest real crowd uses.
- **Three bugs were caught before publication,** each of which would have silently moved text on the page while still painting every letter — the kind of fault that renders fine on the test document and wrong on someone's contract.
- **A real 10-page report was run end to end** and immediately exposed a false positive the synthetic tests could not: the redaction detector was reading letter positions without accounting for the page's own transformation, and flagged two entire pages. Fixed, with a regression test built from the exact transformation a Chrome-printed page opens with.
- **The deep clean grows files by about a third** (448 KB → 590 KB on that report). Since file size is itself one of the clues we report as leaking, that is a real cost and it is now a published limit rather than a footnote.

6.5 Phase 3 milestones

	Deliverable	Status
M0	Groundwork spike; serializer decision taken and recorded	✓ Done
M1	PDF corpus + proof that incremental-update history is provably gone	✓ Done
M2	PDF structure walker, serializer, content tokenizer; recursive F1	✓ Done
M3	Measure which channel actually leaks, before designing the fix	✓ Done
M4	PDF F2 + the honest fingerprint answer, whatever it turned out to be	✓ Done — answer published as a failure, with the floor named
M5	PDF F3 (rasterise) + redaction-risk advisory	✓ Done — including the measurement that disproved the escape route
M6	Word/DOCX walker + F1, with correct ZIP timestamp handling	→ SOON Next
M7	Revision-save IDs measurably cleared where the leading tool leaves them	→ SOON

6.6 What remains in this phase

Word revision-save IDs (RSIDs). Word stamps identifiers throughout a document that link individual edits to editing sessions. Published research shows they **survive every surveyed scrubber, the leading open-source one included**. Clearing them is a capability row we win outright, exactly as M4A was.

One design decision is already settled and is worth stating, because it is the same principle as everything above: a Word file is a ZIP archive, and **a ZIP entry carries a mandatory timestamp that cannot be left out**. Inventing a value there would make our files unique. So we will use the ZIP epoch — **1980-01-01**, the same constant every reproducible-build tool uses — because joining a large existing crowd is what anonymity means, and a value nobody else writes is a signature.

7. A new capability on the table: decoy metadata

This came out of a question asked during Phase 3, and it is worth reporting because it identifies a real gap in the threat model rather than a missing feature.

The question. Since we rebuild a PDF from scratch, could we run the tool inside a **virtual machine with the clock set back**, so the cleaned document comes out with a creation date in the

past instead of today's?

The technical half of the answer: no VM is needed. We own every byte of the output — a creation date is simply a string our own serializer writes, so it can be given any value directly. A rolled-back clock would only help if something in the pipeline were stamping the real time behind our back, and the tool is deliberately built so that nothing does: there is **no wall-clock call anywhere in the scrubber**, which is why our outputs currently carry no date at all. If anything ever did stamp one, that is a bug to fix in the serializer, not something to work around with a virtual machine.

The half that is a genuine finding: this is a different operation from scrubbing, and we do not currently offer it.

	What it gives you
Removing a date (what we do today)	You look like everyone who cleaned a file
Spoofing a date (the proposal)	You look like an ordinary file nobody ever cleaned

Our published adversary ladder asks `"which program made this file"`, and for that question removal is the correct answer. But the proposal identifies a case the ladder does not model: **where the fact of having been cleaned is itself the leak.** A document leaked from a pool of three people, where only one person's files are conspicuously blank, is not protected by being anonymous — it is identified by being anonymous. That is a real scenario and a legitimate reason to want a cover story.

****Why a *random* date would be worse than none. A spoofed date has to be consistent with everything else in the file, and Phase 3 has just finished cataloguing exactly what those things are. A PDF's version number, its use of certain internal structures, its font technology and its embedded photos' own encoder traits each establish an earliest possible date**.** A file claiming 2004 while using structures introduced in 2005 is falsified by reading its first line — and that upgrades "this file was cleaned" (true, harmless) into "this file was cleaned *and* somebody forged its provenance", which reads far worse.

And the generator itself would become a fingerprint. If every file we produce uses the same timezone offset, the same precision, or a uniform spread across a decade while real documents cluster in weekday working hours, then across several files from one user *that distribution is the signature*. This is precisely the class of self-inflicted leak our own fingerprint guard has already caught three times — an empty comment block in FLAC, a fixed padding run, and a compression setting no real producer uses.

So the specification is: constrained, not random. Compute the earliest date the file's own technology allows, then draw from a distribution shaped like genuine human document timestamps above that floor. And it does not ship until an experiment (**E-DECOY**) has measured whether a classifier can separate our synthetic dates from real ones — an unmeasured claim is not something this project publishes.

Where it sits in the plan. This is cross-format by nature: a PDF creation date, a photo's "date taken", an audio track's recording year and a video's creation time all pose the identical question, so it is one shared module rather than a feature per format. It is scheduled as a **Phase 6 integration item, off by default**, on an axis at right angles to the three cleaning levels: those trade **content**, this one trades **truthfulness**.

One flag, stated once and recorded in the limits register when the mode lands. Convincing false provenance on a document has an obvious second use in forgery. Privacy anonymisation is the legitimate use, and that belongs written down as a stated property of the mode rather than left implicit.

8. Far future — the road to the finished tool

The end goal is one tool that irreversibly scrubs files of **arbitrary type**. The remaining phases, in dependency order:

Phase 4 — media and camera formats (MP4 → HEIC → RAW). Video and modern phone photos share the same internal box structure we already built for M4A, so MP4 and HEIC reuse existing code. Camera RAW files are the hardest: they embed a **full-size JPEG preview and a thumbnail, each carrying its own complete set of camera and GPS data** — a major leak, and one our image handlers are already built to clean recursively.

Phase 5 — executables (Windows, Linux, macOS binaries). A separate universe with no shared code: compiler toolchain fingerprints, source-code paths left in debug records, build identifiers, and unique binary IDs. The hard constraint is that the program must still run identically afterwards.

Phase 6 — the long tail and whole-pipeline integration. SVG, TIFF, GIF, WebP, EPUB and similar, most of which reuse the standards modules already written. Then final integration: one dispatcher across every handler, batch processing, and **recursive container cleaning** — a document inside an archive inside an email attachment, cleaned all the way down. **Decoy metadata (§7)** is scheduled here, off by default, once its own experiment has been run.

The parallel deliverable: a professional funding proposal — technical content backed by the measured guarantee matrices, plus a cost plan. Every phase feeds it directly: the matrices **are** the evidence base, which is why they are generated by machine and re-verified on every build rather than written by hand.

The end goal, stated precisely: forensic unrecoverability, validated by differential testing at **every** known hiding place, per format, against the medium-tier adversary — with every residual that cannot be removed documented in plain words rather than quietly omitted.

9. How this project works, and why it is ordered this way

Three principles explain most of the decisions above, and they are worth stating because they are what makes the results trustworthy.

Leaves before containers. File formats nest: a PDF contains JPEGs, a Word document contains images and a rendered preview, a RAW photo contains two JPEGs, a video contains cover art. A container can only be cleaned properly once the formats it embeds can be cleaned. That is why images and audio came first and documents come now — not because they are more popular, but because everything else depends on them. Building PDF first would have forced us to destroy every document's text layer to claim success.

Shared code, written once. The EXIF, XMP, ICC and box-structure modules are written once and called by every format handler. A copy-pasted parser is how a missed update becomes a leak in one format and not another.

No claim without an experiment. Feasibility is expressed as a per-format matrix of what is reachable at which cost — never a binary "yes it's clean". Every cell carries the verdict **and** the reason and residuals behind it. The build re-measures all of it and fails if a published claim has drifted. When we have found our own claims to be broader than our evidence — three times so far, most recently on PDF — we have narrowed them in writing rather than leaving them standing. The Phase 3 flattening result is the clearest example: it would have passed as a win on the obvious test, and we published it as a failure because the harder test says so.

10. Status at a glance

Phases complete	0 (foundation), 1 (images), 2 (audio)
Phase in progress	3 (documents) — PDF complete at all three levels ; Word next
Formats shipping	JPEG, PNG, MP3, FLAC, M4A, PDF
Automated tests	345, run on every change
Published claims	Re-measured from scratch by machine on every build
Known limits	18, published in plain language and rendered into every report
Newly specified	Decoy metadata — a plausible cover story instead of a blank file, off by default, pending its own experiment (§7)
Capabilities no surveyed competitor has	M4A support at all · a PDF clean that is neither a re-render nor an append · lossless-and-untraceable (PNG, FLAC) · user-chosen fidelity · a measured guarantee matrix